

CASO PRÁTICO

CONSTRUÇÃO DE BASE DE DADOS E CONEXÃO COM PHP E A SUA VISUALIZAÇÃO: BANDA/ARTISTA DE MÚSICA

OBJETIVOS

A realização deste caso prático visa o aprofundamento dos conceitos incluídos no módulo “Base de dados MySQL”, tendo em conta os seguintes objetivos de aprendizagem:

- Aplicar os conhecimentos adquiridos ao longo das unidades didáticas do módulo de bases de dados MySQL.
- Modelar uma base de dados.
- Criar tabelas em bases de dados.
- Criar views de tabelas de bases de dados.

ENUNCIADO

Neste caso prático, pretende-se que aplique os conhecimentos adquiridos ao longo das unidades didáticas do módulo de bases de dados MySQL.

Para tal, pede-se que desenvolva uma base de dados relacional em MySQL para gerir eventos de concertos, bilhetes, vendas e clientes. Além disso, deve criar **views** para simplificar a consulta e visualização dos dados.

Deve começar por criar um esquema visual da base de dados relacional para a gestão de venda de bilhetes para concertos. Este diagrama ajudará a visualizar a estrutura e as relações entre as tabelas. Para tal deve:

- Escolher uma ferramenta de modelagem.
- Criar as tabelas (events, tickets, sales, customers).
- Adicionar os atributos para cada tabela.
- Definir as chaves primárias (PK) para cada tabela.
- Adicionar as chaves estrangeiras (FK) para mostrar os relacionamentos.
- Desenhar as linhas de relacionamento entre as tabelas.

Deve criar as seguintes tabelas:

- Tabela **events**: armazena informações sobre os eventos de concertos. Deve incluir:
 - id: identificador único do evento.
 - name: nome do evento.

- ☐ description: descrição do evento.
- ☐ date: data do evento.
- ☐ time: hora do evento.
- ☐ venue: local onde o evento será realizado.
- ☐ capacity: número máximo de lugares disponíveis no local.
- Tabela `tickets`: armazena informações sobre os bilhetes disponíveis para os eventos.
 - ☐ id: identificador único do bilhete.
 - ☐ event_id: referência ao evento ao qual o bilhete pertence.
 - ☐ price: preço do bilhete.
 - ☐ seat_number: número do lugar (se aplicável).
- Tabela `sales`: armazena informações sobre as vendas realizadas.
 - ☐ id: identificador único da venda.
 - ☐ ticket_id: referência ao bilhete vendido.
 - ☐ sale_date: data e hora da venda.
 - ☐ customer_id: referência ao cliente que comprou o bilhete.
- Tabela `customers`: armazena informações sobre os clientes.
 - ☐ id: identificador único do cliente.
 - ☐ name: nome do cliente.
 - ☐ email: email do cliente.
 - ☐ phone: número de telefone do cliente.

Deve ainda criar as seguintes views, para simplificar a consulta dos dados:

- View: `event_details`:

Fornece uma visão consolidada de todos os detalhes sobre eventos, incluindo o número total de bilhetes disponíveis e vendidos.

 - ☐ Campos:
 - event_id: identificador do evento.
 - event_name: nome do evento.

- description: descrição do evento.
- date: data do evento.
- time: hora do evento.
- venue: local do evento.
- capacity: capacidade total do local.
- total_tickets: número total de bilhetes disponíveis.
- sold_tickets: número de bilhetes vendidos.
- available_tickets: número de bilhetes disponíveis restantes.

■ View: `customer_sales_summary`:

Fornece um resumo das vendas realizadas por cada cliente, incluindo o total gasto e o número de bilhetes comprados.

□ Campos:

- `customer_id`: identificador do cliente.
- `customer_name`: nome do cliente.
- `number_of_tickets_purchased`: número total de bilhetes comprados.
- `total_spent`: total gasto pelo cliente.

■ View: `event_sales_summary`:

Mostra um resumo das vendas para cada evento, incluindo o total de receita gerada e o número de bilhetes vendidos.

□ Campos:

- `event_id`: identificador do evento.
- `event_name`: nome do evento.
- `number_of_tickets_sold`: número total de bilhetes vendidos.
- `total_revenue`: receita total gerada pelo evento.

■ View: `tickets_status`:

Fornece uma visão detalhada dos bilhetes, incluindo o status de venda e informações sobre o evento associado.

□ Campos:

- ticket_id: identificador do bilhete.
- event_name: nome do evento associado ao bilhete.
- price: preço do bilhete.
- seat_number: número do lugar (se aplicável).
- status: status do bilhete ('Sold' ou 'Available').

Após criar as tabelas e views, valide e teste para garantir que todos os dados e relacionamentos estão corretos e que as views retornam os resultados esperados.

Deve documentar o propósito de cada tabela e view, bem como qualquer lógica específica utilizada.



Importante

Para a entrega do seu caso prático deverá submeter um arquivo .zip ou similar com todos os arquivos necessários, incluindo o export da base de dados e um PDF com a documentação dos passos seguidos e o esquema criado.

Boa sorte e bom trabalho!

CRITÉRIOS DE CORREÇÃO

Para a correção do seu caso prático, serão considerados os seguintes critérios:

- **Criação do esquema visual da base de dados (10%).**
- **Criação das tabelas (40%).**
 - ☐ Tabela events (6%).
 - ☐ Tabela tickets (6%).
 - ☐ Tabela sales (6%).
 - ☐ Tabela customers (6%).
 - ☐ Adição de atributos e chaves (16%).
- **Criação das views (40%).**
 - ☐ View event_details (10%).
 - ☐ View customer_sales_summary (10%).
 - ☐ View event_sales_summary (10%).
 - ☐ View tickets_status (10%).
- **Validação e testes (10%).**
 - ☐ Validação das tabelas e relacionamentos (5%).
 - ☐ Testes das views (5%).

